

Accessibility — Interview Cheatsheet

Mnemonic-first. Cram page 1; the rest is the lookup table behind it. Dark companion to React/CSS · Coding Patterns · JS · System Design.

FRANK — would Frank get through this?

Frank is your only user who can't touch the mouse and can't see the screen. Run every component past him. Five letters = the bare minimum any FE interviewer is checking for.

- F Focus** — visible ring on every control (never bare `outline:none`), and focus is *moved* into modals / *restored* to the trigger on close / sent to the new `<h1>` on route change.
- R Role** — the right semantic element (native first: `button`, `a href`, `nav`). Its **state** attrs (`aria-expanded` / `selected` / `checked` / `current`) match reality *and update on change*.
- A Announce** — anything that changes without a reload (errors, toasts, "saved", streaming replies) is spoken via a **live region**.
- N Name** — every interactive control has an accessible name. Icon-only button → `aria-label`. A screen reader must not just say "button".
- K Keyboard** — reach *and* operate everything with keys alone. Nothing mouse-only. This is the one they actually test in the coding round.

Tab between, Arrows within

Tab jumps *across* widgets; arrow keys move *inside* one composite (menu, tabs, radio group, listbox). A whole widget = one Tab stop.

Enter + Space = button

Any `role="button"` you hand-roll owes *both* key handlers, and `preventDefault` on Space (else the page scrolls).

Polite streams, assertive screams

Live regions default to **polite** (waits for a pause). Reserve **assertive** for genuine must-interrupt errors. A streaming reply is *always* polite.

Move vs Mark

Roving tabindex **moves** real DOM focus item-to-item. `aria-activedescendant` **marks** the active child while focus stays on the container (combobox).

60 seconds before the round? Say "FRANK" to yourself and the four rules above. That's the pass mark. Everything on pages 2–7 is you *exceeding* it.

The recall model + roles you should know cold

You never recall `aria-current` in isolation — you recall it from the **widget it belongs to**. Anchor every attribute to a pattern; that's the whole trick. Roles fall into 4 buckets.

Landmark roles → native equivalent

Role	Native tag
banner	top-level <header>
navigation	<nav>
main	<main> (one/page)
complementary	<aside>
contentinfo	<footer>
region	named <section>

Two navs? Distinguish with `aria-label` ("Primary" / "Footer").

Live-region roles (implicit live semantics)

- `alert` = assertive + atomic → **errors**
- `status` = polite + atomic → **"saved" / toasts**
- `log` = polite, non-atomic → **chat message list**

Recall trick: alert screams, status states, log logs. Match the noun to the urgency.

Widget roles (composite ones carry required keyboard)

Role	Required state
button	Enter+Space if non-native
checkbox/switch	<code>aria-checked</code>
radiogroup/radio	arrow-key roving
tablist/tab/tabpanel	<code>aria-selected+controls</code>
menu/menuitem	arrows + typeahead
listbox/option	<code>aria-selected</code>
combobox	<code>expanded+controls</code>
dialog/alertdialog	<code>aria-modal="true"</code>
slider/progressbar	<code>aria-valuenow/min/max</code>
tree/treeitem	<code>aria-expanded</code> on nodes

Fast triage per component

Ask	Reach for
Known widget?	APG role + keymap (p.4)
Shows/hides content?	trigger trio (p.3)
Changes live?	live region (p.3)
Arrow-navigable list?	Move vs Mark (p.5)

ARIA states & properties — grouped by why

Memorize by *cluster*, not alphabetically. On a nav item you reach for the disclosure/selection cluster and `aria-current` falls out.

Naming & description — mnemonic: a pointer beats a string beats the text beats a placeholder

Attr	When
<code>aria-labelledby</code>	Name = other visible element(s), by id. Wins over everything.
<code>aria-label</code>	Icon-only control; a raw string. Silently overrides visible text.
<code>aria-describedby</code>	Extra hint/error, read <i>after</i> the name.
<code>title/placeholder</code>	Last resort — not a real label.

Disclosure & selection — nav · tabs · menus · accordions · comboboxes

Attr	Values	Meaning
<code>aria-current</code>	page · step · location · date · time · true	The "you are here" item in a set. Nav link → page (your default recall); wizard → step .
<code>aria-expanded</code>	true / false	On the <i>trigger</i> — is the controlled thing open?
<code>aria-controls</code>	id	Trigger → the panel/menu/listbox it toggles.
<code>aria-haspopup</code>	menu · listbox · dialog · true	Trigger opens a popup of that kind (true=menu).
<code>aria-selected</code>	true / false	option / tab / row selection.
<code>aria-checked</code>	true / false / mixed	checkbox / radio / switch.
<code>aria-pressed</code>	true / false / mixed	Toggle button (mute, bold).

Trigger trio — any show/hide control carries all three together: `aria-expanded` + `aria-controls` + (if a real popup) `aria-haspopup`.
checked vs pressed: check-like things get checked, buttons get pressed.

Live regions — "polite streams, assertive screams"

Attr	Values / meaning
<code>aria-live</code>	polite · assertive · off
<code>aria-atomic</code>	true=re-read whole region · false=just the change
<code>aria-relevant</code>	additions · removals · text · all
<code>aria-busy</code>	true while loading (suppress partials)

Gotcha: the region must be in the DOM *before* you inject text, or nothing is announced. Render an empty `<div aria-live="polite">` on mount, then fill it.

Forms, range & misc

Attr	Meaning
<code>aria-invalid</code>	true/false/grammar/spelling
<code>aria-errormessage</code>	id of error text (pair w/ invalid)
<code>aria-disabled</code>	disabled but still focusable
<code>aria-valuenow/text</code>	slider / progress value
<code>aria-setsize+posinset</code>	virtual lists ("12 of 500")
<code>aria-hidden="true"</code>	drop decorative node from a11y tree

disabled vs aria-disabled: native `disabled` drops it from tab order (undiscoverable); `aria-disabled` keeps it focusable so users find it, then explains why it's off.

Keyboard maps (WAI-ARIA APG) — "Tab between, Arrows within"

The ones interviewers expect you to reproduce. Tab moves *between* widgets; arrows move *within* one composite widget.

Key	Action
Tab	Into tablist (active tab), then out to panel
← →	Prev/next tab (roving)
Home End	First / last tab
Enter Space	Activate (manual mode only)

Menu / menu button

Key	Action
Enter Space ↓	Open, focus first item
↑ on btn	Open, focus last item
↑ ↓	Move between items
a–z	Typeahead to match
Esc	Close, focus back to button
Tab	Close menu, move on

Radio group

Key	Action
Tab	Into group (checked, or first)
↑ ↓ ← →	Move and select (roving)

Key	Action
on open	Focus into dialog (first field / dialog)
Tab / ⇧Tab	Cycle within — focus trap
Esc	Close
on close	Restore focus to trigger

Combobox / listbox

Key	Action
↓ / ↑	Open + move through options
Alt+↓	Open without moving
Enter	Commit highlighted option
Esc	Close / clear input
type	Filter; focus stays in input → <code>activedescendant</code>

Accordion / disclosure

Key	Action
Enter Space	Toggle panel (flips <code>aria-expanded</code>)

Grid (2-D)

Key	Action
↑ ↓ ← →	Cell to cell
Home / End	Row start / end
Ctrl+Home/End	First / last cell

Focus management — "Move vs Mark"

Every arrow-key list uses one of these two. Say the name, then the tradeoff — the single highest-signal a11y topic in FE coding rounds.

Move → roving tabindex

Exactly one item is `tabindex="0"`; all others `tabindex="-1"`. On arrow, change which is 0 and call `.focus()`. **Real DOM focus moves.**

Use when the items themselves take focus: toolbars, menus, tabs, radio groups, tree.

```
// active = index whose tabindex is 0
function onKeyDown(e) {
  const last = items.length - 1;
  let next = active;
  if (e.key === "ArrowDown") next = active === last ? 0 :
active + 1;
  else if (e.key === "ArrowUp") next = active === 0 ?
last : active - 1;
  else if (e.key === "Home") next = 0;
  else if (e.key === "End") next = last;
  else return;
  e.preventDefault();
  setActive(next);
  refs.current[next]?.focus(); // move real focus
}
// render: tabIndex={i === active ? 0 : -1}
```

Why one 0, rest -1? Keeps the widget a **single Tab stop** — Tab skips past it, arrows navigate inside. All -1 = still focusable by JS but skipped by Tab.

Mark → aria-activedescendant

The **container** holds DOM focus (`tabindex="0"`). Children have `ids`, no `tabindex`. You point `aria-activedescendant` at the "focused" child; SR announces it, **focus never leaves the container.**

Use when focus must stay put — **combobox**: focus stays in the input as you arrow through options.

```
<input role="combobox" aria-expanded="true"
  aria-controls="lb"
  aria-activedescendant="opt-2" />
<ul id="lb" role="listbox">
  <li id="opt-2" role="option"
    aria-selected="true">...</li>
</ul>
```

	Move	Mark
DOM focus	moves to item	stays on container
Best for	toolbar/menu/tabs	combobox/grid
You manage	<code>tabindex + .focus()</code>	<code>id</code> stays in sync

- `tabindex="-1"` = focusable by script, skipped by Tab (skip-link target, route-change heading).
- Never `tabindex > 0` — it hijacks the page's whole tab order.

keydown recipes + component builds

Custom button — "Enter + Space = button"

```
<div role="button" tabIndex={0}
  onClick={act}
  onKeyDown={e => {
    if (e.key === "Enter" || e.key === " ") {
      e.preventDefault(); // Space would scroll
      act();
    }
  }}>Save</div>
```

Native `<button>` does all this free — say that first, then show you *can* do it by hand.

Escape to close / typeahead

```
if (e.key === "Escape") { close();
triggerRef.current?.focus(); }

// typeahead: jump to first item starting w/ key
if (/^[a-z]$/i.test(e.key)) {
  const i = items.findIndex(x =>
    x.toLowerCase().startsWith(e.key.toLowerCase()));
  if (i >= 0) focusItem(i);
}
```

Focus trap (modal) — essence

```
function trap(e) {
  if (e.key !== "Tab") return;
  const f = dialog.querySelectorAll(
    'a[href],button,input,select,textarea,' +
    '[tabindex]:not([tabindex="-1"])');
  const first = f[0], last = f[f.length-1];
  if (e.shiftKey && document.activeElement === first) {
    e.preventDefault(); last.focus();
  } else if (!e.shiftKey && document.activeElement ===
  last) {
    e.preventDefault(); first.focus();
  }
}
// open: save activeElement, focus first field
// close: restore saved element.focus()
```

Name the shortcut: native `<dialog>.showModal()` gives the trap, `Esc`, and inert backdrop for free.

Streaming chat a11y — your double-star build

A streaming reply is a **live-region** problem. "Polite streams" — never assertive.

```
// message log: announce new content politely
<div role="log" aria-live="polite"
  aria-relevant="additions">
  {messages.map(m =>
    <div key={m.id}>
      <span className="sr-only">{m.role}</span>
      {m.text}
    </div>)}
</div>

// "typing" status (separate polite region)
<div role="status">{streaming ? "Assistant is
responding" : ""}</div>
```

- **polite, not assertive** — assertive on every token = torture.
- `role="log" + aria-relevant="additions"` announces only new text, not the whole growing message.
- Input: real `<label>/aria-label`; `Enter` sends, `Shift+Enter` newline.
- Decorative avatars/icons → `aria-hidden="true"` or `alt=""`.

Tabs — the required wiring

```
<div role="tablist" aria-label="Settings">
  <button role="tab" id="t1"
    aria-selected="true" aria-controls="p1"
    tabIndex={0}>General</button>
  <button role="tab" id="t2"
    aria-selected="false" aria-controls="p2"
    tabIndex={-1}>Billing</button>
</div>
<div role="tabpanel" id="p1"
  aria-labelledby="t1" tabIndex={0}>...</div>
```

Definition of done + gotchas that read as a fail

The pass = run FRANK out loud at the end

- **F — Focus:** visible ring; trapped + restored in modals; sent to new heading on route change.
- **R — Role:** correct role; `aria-expanded/selected/checked/current` reflect + update.
- **A — Announce:** toasts / errors / streaming via live regions.
- **N — Name:** every control named (icon buttons especially).
- **K — Keyboard:** Tab / arrows / Enter / Space / Esc all work; nothing mouse-only.
- **+ Contrast:** text $\geq 4.5:1$ (large / UI $\geq 3:1$); never color as the *only* signal.

How you'd verify (say this)

- Tab through it, mouse untouched.
- Axe / Lighthouse for the automated ~30%.
- Screen-reader spot check (VoiceOver ⌘+F5 / NVDA).
- "Automated tools catch maybe a third — keyboard + SR catch the rest." Interviewers like that framing.

Gotchas (each is a common ding)

- **Placeholder as label** — vanishes on type, not a name. Use `<label>`.
- **div+onClick, no keyboard** — mouse-only. Use `<button>`.
- **outline:none** with no `:focus-visible` replacement.
- **Focusable child inside aria-hidden** — SR-invisible tab trap.
- **Live region injected with its text** — must pre-exist in DOM.
- **aria-label overriding visible text** — breaks voice-control users.
- **tabindex > 0** — reorders the whole page.
- **Color-only state** — add text/icon (~8% of men are color-blind).
- **Redundant roles** — `<button role="button">` is noise.

If you forget everything else: FRANK + Tab between / Arrows within + Polite streams / assertive screams + Move vs Mark.
Five letters and three lines clear the bar in any FE loop.